**CS 4398 Software Engineering Project**

**Project Title: Automated Warehouse Restock System (AWRS)**

**Group 4**

Aya Alkhatib (mjl171@txstate.edu)

Jade Wilson (qgy13@txstate.edu)

Tatenda Machirori (qgr28@txstate.edu)

Zainab Khan (dov21@txstate.edu)

**Table of Contents**

1. Introduction

1.1 Purpose of the SRS

1.2 Problem Statement

1.3 Intended Audience and Document Conventions

1.4 Product Scope

1.5 References

2. Statement of Functional Requirements

(... organized by user tasks / key workflows ... Corresponding use-case/class/sequence diagrams attached where appropriate. GUI screenshots of envisioned WP forms/dashboards included as we prototype in Qt.)

3. Non-Functional Requirements

4. Design/Implementation Constraints

5. References and Sources of Information

# 1. Introduction

## 1.1 Purpose of the SRS

This document is the Software Requirements Specification (SRS) for the Automated Warehouse Restock System (AWRS). It lays out the functional and non-functional requirements for a desktop application to track warehouse inventory and automate warehouse restocking functions, targeting small-to-medium warehouse businesses. It acts as the core reference for development, testing/validation, and future maintainers/extenders of the application to ensure all stakeholders are on the same page.

## 1.2 Problem Statement

Existing manual processes of tracking inventory in small-to-medium warehouses lead to many problems: stockouts, overstocking, human error, delays in triggering restocking work, inability to forecast and be proactive. We will solve this by enabling warehouse managers to easily track inventory at specific locations in real-time, automatically identify when inventory is low and trigger restock tasks, prioritize restock tasks, include basic prediction to forecast when an item will be depleted based on historical patterns to be proactive vs reactive, provide role-based authentication/access, dashboards/alerts/reports, and maintain full audit logging/trail of changes. This will all be done in a simulated environment with no integration to any actual hardware (only GUI-based inputs for actions such as receiving items, scanning locations, picking for orders, etc.) though we will architect the solution to allow for extensibility points later.

## 1.3 Intended Audience and Document Conventions

- Intended audience: Team members responsible for development (Group 4), course instructors/evaluators, future maintainers (see item below), and project stakeholders such as end-users who may desire to extend functionality (e.g. warehouse operations managers who want to deploy this software for their business).

- Conventions: Requirements shall be stated in active voice with the word "shall" to indicate a requirement. Relevant UML diagrams will be added for key classes/use cases/sequences/statechart, etc. Screenshots of GUI forms will be provided from our prototyping in Qt. Acronyms: AWRS, SKU, Qt.

### 1.4 Product Scope

AWRS is a desktop application software to track inventory and automatically trigger/complete restocking of inventory in warehouse facilities. Developed in C++17 with Qt 6.x GUI and SQLite database, it will allow for multiple warehouses, each with configurable hierarchical locations down to the shelf/bin level, perform core warehouse workflows (receiving, fulfillment/picking/packing, adjustments, automated restocking), support role-based authentication/access (admin, warehouse manager, worker roles/tiers), include "smart" restocking engine (triggered by rules-based thresholds, include lightweight predictive triggers based on historical usage patterns), include warehouse dashboards + alerts/notification and reporting, and maintain full audit logs of transactions. This will be a simulated application (no actual integration to any hardware like barcode scanners, warehouse robots, etc.) though architected to allow for extensibility points later for future hooks into ERP systems, hardware devices, etc. Applications outside of scope include: mobile/web access (only desktop application), lack of advanced ML-based prediction forecasting, external vendor APIs, and enterprise-scale distributed database.

### 1.5 References


- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications.

- Project description document for AWRS (provided by course instructors for our team).

- Qt 6.x Documentation. https://qt.io

- SQLite Documentation. https://sqlite.org

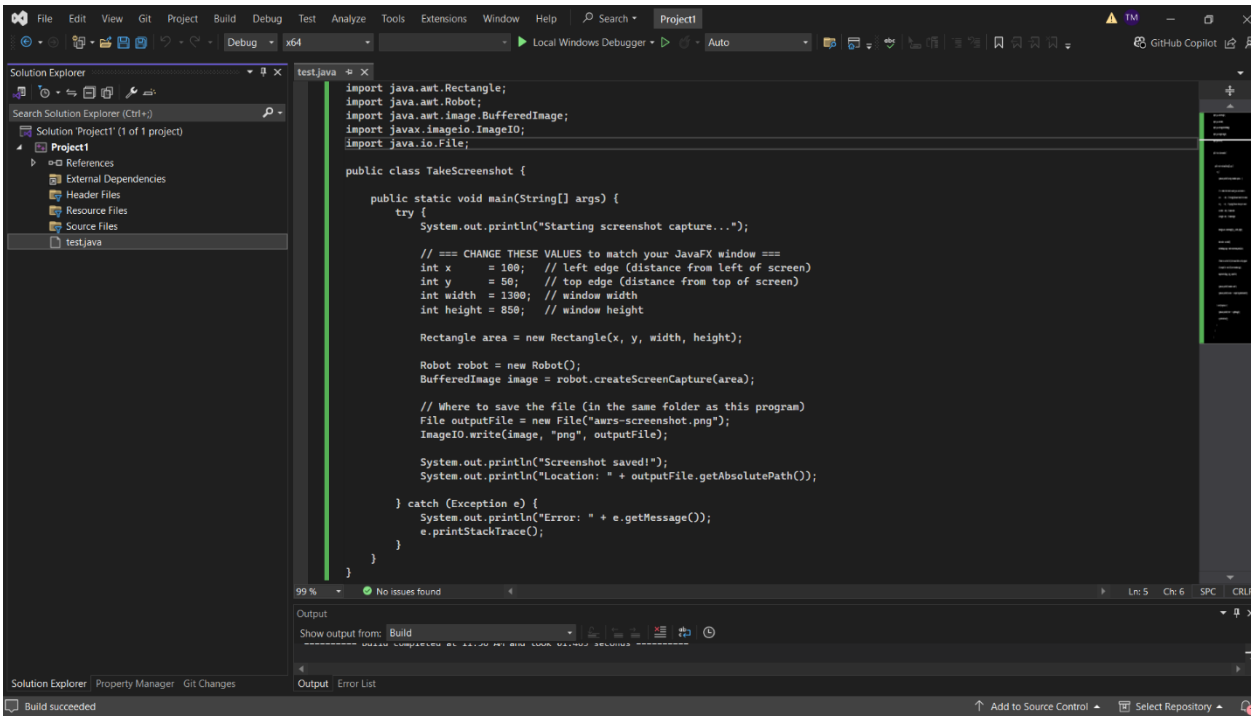

## 2. Statement of Functional Requirements

Our functional requirements will be grouped by key user tasks/workflows which largely correspond to our 4 primary workflows/features of the application. Each functional requirement will have accompanying text description (included here), any associated roles that interact with the feature, and links to our use-case/class/sequence diagrams (to be added) and GUI screenshots captured from our QT prototyping (also to be added).

### 1. User Authentication and Role-Based Access Control

- Users shall be required to login with proper credentials and shall be restricted to roles assigned to their account (Admin users have full-system access, Manager accounts have oversight/config capabilities, and Worker users are limited to completing tasks)

- Admin users shall be able to create, edit, delete, and assign roles to other users.




## 2. Item Catalog and Location Definitions

- Users shall be able to enter new items (receive) into inventory by specifying common attributes (SKU, description, supplier, unit of measure)

- Admin and Manager level users shall be able to define warehouses and locations within a hierarchical tree structure (e.g. aisle > shelf > bin)

- System shall keep track of inventory counts of each item at each location, updating in realtime as transactions occur.

## 3. Receive Shipments

- Users shall be able to input receiving information of new shipments -> update quantity of item at given location -> automatically create audit log entry of transaction.

## 4. Fulfill Orders (Order picking/packing)

- Users shall scan/make entries for items to fulfill outgoing orders, automatically deduct quantity from location -> low-stock check triggered.

**5. Inventory Adjustments**

- Users shall be able to make manual inventory adjustments for reason codes including damage/loss/etc. and require Manager or Admin level approval to submit.

**6. Automated Restocking Engine**

- System continuously monitors a configurable set of rules per item/location (minimum/maximum thresholds, reorder point, supplier lead time) daily/based on triggered events.

- If item is below minimum required, automatically create restock tasks with priority determined by distance from reorder point / estimated time until stockout. Create alerts/dashboard notifications for pending tasks.

- Worker users shall be able to view pending restock tasks, can be assigned to specific workers, and marked complete when item is physically replenished at proper location (another trigger to update warehouse inventory counts).

- Batch process algorithm should support processing many restock tasks at once.

**7. Predictive Analytics**

- System shall evaluate past history of item usage at a location and forecast when the inventory will likely be depleted (simple moving average calculation + growing/shrinking trend) and create alerts/dashboard notifications to restock BEFORE low-stock threshold is reached.

**8. Monitoring/Dashboards and Reporting**

- Allow for dashboard with at-a-glance view of stock levels/health, visual alerts, etc. and view pending restock tasks.

- Include generation of reports such as stock movement, stockouts avoided, top active items, predictive analytics alerts.

- Audit logs shall maintain full history of transactions indefinitely.

### *3. Non-Functional Requirements*

**Performance**

- Response time for typical transactions such as updating/querying inventory < 2 seconds on datasets consisting of up to 10,000 total items/spots. Optimized to scale to medium workloads via SQLlite3, but would require stress/performance tests if used with large warehouses/inventory counts.

**Security**

- Permissions enforced via role-based access control (RBAC) model. All users must be authenticated. All changes to inventory shall be logged to full audit trail.

**Usability**

- GUI built via Qt with dashboards/alerts/widgets, forms for common workflows, and visual display of tables for tracking items/locations. Ease of use for warehouse workers is a priority, assuming only minimal training is provided.

**Maintainability**

- Adopted clean code practices such as separation of concerns (presentation, database, and business logic layers), use of common design patterns (Observer, Strategy, Factory, Singleton patterns), internal commenting and documentation, and loosely coupled architecture to allow for ease of maintenance and extensibility.

**Reliability**

- Inventory quantities shall always be accurate based on transactions entered. Transactions are processed via SQLite on a transactional basis (commit/rollback) to ensure integrity. Inputs are checked for invalid data.

**Portability**

- Desktop application should be runnable on Windows/Linux/macOS machines with C++17 and Qt.

## 4. Design/Implementation Constraints

- This SRS is written in accordance to guidelines from IEEE 830-1998.

- We will be using C++17 as our programming language.

- QT 6.x will be used to design our GUI (Qt widgets).

- SQLite will be used for our database, through the Qt SQL module.

- VSCode will be used as our IDE with CMake as our build system.

- Git + GitHub for version control, with continuous integration done via GitHub Actions.

- Google Test Library (https://github.com/google/googletest) will be used for unit testing code segments, and Qt Test for integration and GUI tests.

- UML diagrams will be created via draw.io (https://app.diagrams.net). No official standards were provided so we are following general UML standards (OMG).

- Agile/iterative development methodology (Split into 3 iterations). TDD development of core application logic where possible. Pair programming/code reviews.

- There will be no actual integrations to hardware (simulate all manual workflows via GUI input forms). Stock updates triggered by buttons instead of actual scanners, restocking quantities entered via GUI instead of triggered by robotics, etc.

- There will be no external integrations in initial project scope. No hooking into any ERP APIs.

## 5. References and Sources of Information


- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications.

- Project description and details for AWRS (sent to our group by TXState instructors).

- Qt 6 Documentation. qt.io

- SQLite Documentation. sqlite.org

- Google Test Documentation.

- UML Standards (OMG).